

NT@B Software Division — EEG Motor Imagery Project

Full Reasoning, Decisions, and Plan

Purpose of this document: a complete, self-contained record of what this project is, why every decision was made the way it was, what's been verified versus what's still open, and what to say in the interview and demo video. Written so a mentor, a different AI chat, or you six months from now could read it cold and understand every consideration without needing the original conversation.

Deadline: Friday 9/11, 11:59 PM. **Submission:** <https://forms.gle/QnHSTt8xd4y3HXMd9> **Time budget:** 5-10 hours per the brief. A tight 6-hour submission beats a sprawling 20-hour one; if you pass 10 hours, stop and write up where you are. **Status:** Passed Software division first round; this is the technical project for that round.

1. What This Project Actually Is (Ground-Up Explanation)

1.1 The signal being measured

EEG (electroencephalography) uses electrodes on the scalp to pick up the summed electrical activity of large groups of neurons firing together, filtered through skull and skin. It is not a picture or a video — it's 64 independent voltage traces over time, one per electrode, sampled 160 times per second. Poor spatial resolution, decent temporal resolution. It reads a smeared population signal, not individual neurons.

1.2 The specific phenomenon being classified

Motor imagery is imagining a movement without performing it. Imagining left-hand movement produces a measurable dip in signal power in the mu/beta frequency band (roughly 7-30 Hz) over the *opposite* side of the brain — left hand imagined → right hemisphere dip, and vice versa. This crossover is called ERD (event-related desynchronization). It is a real, physical phenomenon, not a statistical artifact, and it is the actual thing being decoded. This is distinct from eye movement artifacts (EOG) — different signal, different artifact type, worth being able to state clearly if asked.

1.3 The dataset

PhysioNet EEGMMIDB: 109 subjects, 64 channels, 160 Hz, recorded with the BCI2000 system.

Original paper: Schalk et al., IEEE Trans. Biomed. Eng. 51(6):1034-1043 (2004)

Resource paper: Goldberger et al., Circulation 101(23):e215-e220 (2000)

Fetches via `mne.datasets.eegbci.load_data(subject, runs)` — no bundled subset, no starter notebook, you fetch it yourself

Run structure (14 runs per subject):

Runs	Content
1, 2	Baseline, eyes open/closed — not used
3, 7, 11	Executed left vs. right fist
4, 8, 12	Imagined left vs. right fist — this project's task
5, 9, 13	Executed both fists vs. both feet
6, 10, 14	Imagined both fists vs. both feet

Annotation gotcha (explicitly flagged in the brief): within each file, T0 = rest, T1 and T2 mark the two conditions, but **what T1/T2 mean depends on the run type**. In runs 4/8/12 (the ones this project uses), T1 = left, T2 = right. In runs 6/10/14, T1 = fists, T2 = feet. Same label names, different meaning. Because this project only touches 4/8/12, the meaning never flips mid-code — but you should be able to state this gotcha out loud in the interview, since the brief explicitly says to catch it yourself.

Trial counts (verified across multiple published sources, with some natural variability by subject): each two-minute run contains roughly 14-15 trials of rest interleaved with roughly 7-8 trials each of left and right (some papers report ~21 trials per class per subject across all three runs combined; a few subjects have known anomalies in this pattern — worth confirming empirically once the data is loaded rather than trusting any single published count).

2. What Is Actually Being Graded (Direct From the Brief's Own Language)

Quoting the brief's own framing of what it's assessing, condensed to the essential claims:

Your data processing pipeline (filtering, noise handling)

What your model can and cannot do — the range, not the best case

That your result isn't an artifact of how you measured it

What the appropriate baselines are (chance is a weak reference point)

How much of the result is about the *person* rather than the *task*

Whether the model learned or memorized

Critically, the brief states outright: "We do not rank submissions by accuracy." It also states the chance baseline is 50% and warns that your first accuracy figure will look better than the honest one — explaining that gap is described as "most of the assignment."

This reframes the entire project. The task is not "build the best classifier." The task is "build a defensible classifier and then interrogate your own result until you know exactly what it does and doesn't prove." A submission with a modest, fully-interrogated result is explicitly stated to be equal or superior to one with a high, unexamined number.

3. The Method: CSP + LDA

3.1 What each piece is, from zero

CSP (Common Spatial Patterns): a spatial filtering algorithm that finds a small number of weighted combinations of the 64 electrodes that maximize the variance difference between left-trials and right-trials. It is purpose-built for exactly this kind of two-class oscillatory-power problem.

LDA (Linear Discriminant Analysis): draws a linear decision boundary in the CSP-reduced feature space.

3.2 Why this over a CNN, UNet, or YOLO

A mentor conversation (screenshots reviewed) suggested a CNN or UNet, reasoning that convolutional networks are proven for classifiers and worrying about GPU/compute limits on a MacBook. This advice does not transfer to this project, and here's the reasoning:

CNNs and UNets are built to find spatial patterns in a *grid* (like pixel neighborhoods in an image). EEG here is not a grid — it's 64 independent time-series with a known physical relationship (a dip in a specific frequency band, on a specific side, at a specific time). CSP is the technique purpose-built to isolate exactly that structure.

YOLO is an object-*detection* architecture solving spatial localization — "where in this image is the thing." This task has no localization sub-problem: trial boundaries are already given by the T1/T2 annotations. There's nothing left for a detector to detect.

CSP+LDA trains in seconds to minutes on a laptop CPU. No GPU needed, no multi-day training risk, no wifi-dependent background job. The mentor's compute-bottleneck concern was valid advice for a different (image-based) problem, not this one.

A CNN (EEGNet, ShallowFBCSPNet — the actual validated architectures for *this specific data type*) remains a legitimate stretch goal after the core evaluation is solid, not a replacement for it.

3.3 Exact parameters, verified against MNE's own official reference implementation for this exact dataset

Rather than choosing parameters from general knowledge, the following were cross-checked against MNE-Python's own published example code that runs CSP+LDA on this exact PhysioNet dataset (not a generic tutorial — the dataset-specific one):

Bandpass filter: 7-30 Hz. (Note: one other published paper curating this same dataset uses 8-30 Hz instead — both are defensible mu/beta-band choices; 7-30 was chosen here specifically because it matches the library's own reference implementation for this exact dataset, which is the more citable justification.)

Epoch window: -1.0 to 4.0 seconds relative to the cue (`[tmin=-1.0, tmax=4.0]`), with the actual classifier training window narrowed to **1.0-2.0 seconds post-cue** specifically to avoid the sharp onset transient at the cue itself, which is a different signal from the sustained imagined-movement ERD.

CSP: 4 components, no regularization, log-variance features (`[CSP(n_components=4, reg=None, log=True, norm_trace=False)]`).

Classifier: LDA, standard configuration.

Reference "naive" cross-validation: 10-fold Monte Carlo shuffle split, 80/20 (`[ShuffleSplit(10, test_size=0.2, random_state=42)]`) — this is, notably, exactly the kind of naive random split that this project's evaluation gauntlet requires you to compute first and then discredit.

Why this matters for the interview: you can honestly say "I matched my preprocessing and model parameters to MNE's own dataset-specific reference implementation, not generic defaults," which is a stronger, more specific answer than an appeal to "standard practice."

4. The Evaluation Gauntlet (The Actual Assignment)

Six numbers, reported together, each with a distinct purpose:

Naive random-split accuracy. The tempting, easy number — computed specifically to be the wrong answer you then dismantle. Trials from the same subject can land on both sides of the split, letting the model partially shortcut via subject-specific noise fingerprints (skull shape, skin conductance, electrode placement) rather than the actual left/right signal.

Leave-subject-out accuracy. The honest primary number — train on some subjects, test only on entirely unseen ones. This is how the model would actually need to work in deployment.

Same-subject upper bound. Train and test within one person's own data. Establishes the *ceiling* — how much left/right signal exists at all once the cross-person confound is removed. Without this, a modest leave-subject-out number looks like failure; with it, you can show whether you're close to or far from the best achievable result.

Label-shuffle control. Scramble the labels, rerun the identical pipeline. Should collapse to ~50%. If it doesn't, that's a bug-detection tool pointed at your own code (a broken split, information leaking through preprocessing), not a finding about the brain.

Subject-identity leakage check. Point the same feature pipeline at predicting *which subject* a trial came from instead of left/right. If that's easy, it directly measures and explains why the naive split score (#1) was inflated — turning suspicion into a measured quantity.

Chance/majority-class baseline. The floor, reported for completeness, explicitly flagged in the brief as a weak reference point on its own.

The core insight tying these together: a standard train/test split already keeps individual trials separate — that box is checked in the naive split. But "different trials" is not the same as "different information." The real requirement is that the test set differ along *every axis that could leak the answer*, not just the axis you were told to split on. Trial-level splitting catches literal memorization of a snippet; subject-level splitting (#2)

catches the deeper failure of memorizing a *person* well enough to shortcut the actual task. This generalizes: the same trap appears in a multi-hospital medical imaging model randomly split by image instead of by hospital, or any dataset where one entity contributes multiple correlated data points.

5. The Differentiator: Euclidean Alignment (Researched, Not in the Original Brief)

Important distinction, stated plainly: EEGNet and ShallowFBCSPNet, as stretch-goal architectures, are *already named in the brief itself* as a "legitimate stretch goal." Using them as a differentiator doesn't set you apart — it's simply following the brief's own suggestion. Riemannian geometry methods, `pyriemann`, MOABB, and Euclidean Alignment were **not** in the original brief — these came from independent research and are the genuine addition.

5.1 What it is and why it's different in kind from a fancier classifier

The core confound this whole assignment is built around is that each subject's brain signal sits in a slightly different statistical "place" — different skull geometry, different electrode contact, different baseline noise. Euclidean Alignment (EA) doesn't try to build a smarter classifier that shrugs this off; it intervenes one step earlier, as a preprocessing step:

For each trial, compute its covariance matrix (a 64×64 table capturing how electrodes move together over that snippet).

For each subject, average their covariance matrices across all their trials — a "reference matrix" representing that person's typical baseline.

Compute the inverse square root of that reference matrix — a specific, well-defined normalizing operation.

Apply it to every one of that subject's trials before any classifier sees the data.

Only then run CSP+LDA as before, on the now-aligned data.

Why this is a stronger contribution than swapping in a different model architecture: it directly targets the exact question the brief already asks ("how much of this is about the person rather than the task... quantify how much this matters") — by intervention, not just measurement. Reporting the leave-subject-out number with and without alignment turns "I measured the confound" into "I measured it and tried to correct for it, here's what happened."

5.2 Caveats, stated honestly (from the literature, not assumed)

Published comparisons are mixed: some studies find CSP outperforming plain Riemannian covariance methods on certain benchmarks; alignment methods in general are reported to help but not fully close the cross-subject gap (one paper on a different dataset/benchmark reported baseline cross-subject accuracy around 30-45%, improving to roughly 50-53% with Riemannian alignment — **these numbers are from a different dataset and task, not this one, and should not be treated as a prediction for what this project's run will produce**).

Alignment computes each subject's reference matrix using *all* of that subject's trials, including ones later held out as test trials. This is standard/accepted practice in the alignment literature since it's unsupervised (no labels used), but it's a real design subtlety worth being able to name directly if questioned, rather than presenting it as beyond objection.

5.3 Scope decision: is it worth building alongside the required method?

No, not as a full second model or architecture comparison. The brief explicitly does not rank by accuracy, explicitly gives a 5-10 hour budget, and explicitly says to stop and write up where you are past 10 hours. Building multiple full classifier architectures (CSP+LDA vs. a Riemannian classifier vs. EEGNet) is scope expansion with real time risk and no reward from the grading criteria as stated.

Yes, worth including as a small toggle. Euclidean Alignment is a few lines of matrix math inserted as a preprocessing function, not a second pipeline. Estimated cost: 30-60 minutes. It deepens an evaluation question the brief already asks, rather than adding a new one.

Priority order:

Non-negotiable: one classifier (CSP+LDA), all six evaluation checks, fully working, honestly written up. This alone satisfies the entire assignment.

Worth it if the core is solid with time to spare: add the alignment toggle, report leave-subject-out with and without it.

Skip unless far ahead of schedule: a full second architecture run in parallel.

6. Tools Referenced (What's From the Brief vs. External Research)

Named directly in the brief, under "useful tooling, optional":

MNE-Python — the standard EEG/MEG analysis library; does the filtering, epoching, and has the built-in fetcher `mne.datasets.eegbci.load_data(subject, runs)` for this exact dataset.

MOABB — wraps this dataset as a standardized benchmark (`PhysionetMI`). Useful as an external reference to sanity-check the hand-built leave-subject-out logic against — not to build the evaluation inside of, since building it yourself is the point.

Braindecode — a deep learning library for EEG; relevant only if pursuing the EEGNet/ShallowFBCSPNet stretch goal.

Brought in from external research, not in the brief:

pyriemann — Python library implementing Riemannian-geometry classifiers (Tangent Space Mapping, Minimum Distance to Mean), plugs into the same scikit-learn pipeline style as CSP+LDA.

Euclidean/Riemannian Alignment — the preprocessing technique described in Section 5.

A note on VS Code: a mentor's suggestion, referenced in reviewed screenshots. It's a text editor, not something that affects the grading criteria. Reasonable to use, not a strategic factor.

7. A Transparency Note: Corrections Made During Planning

In the interest of this document being fully honest for interview purposes, two things were initially stated with more confidence than was warranted, then corrected:

The frequency band was initially stated as "8-30 Hz" from general mu/beta neuroscience knowledge, without checking this specific dataset's reference implementation. On checking, MNE's own official example for this exact dataset uses 7-30 Hz (a different published paper curating this same dataset does use 8-30 Hz — both appear in real published work, but 7-30 was adopted here as the more directly citable, dataset-matched choice).

Illustrative accuracy numbers (e.g., "naive split might read ~85%, leave-subject-out might come back ~65%") were invented purely to explain the *shape* of the leakage problem conceptually, not sourced from any actual run or published result on this exact task. These should be discarded entirely once real numbers are produced — they were never predictions.

This is included here deliberately: being able to say "here's a number I second-guessed myself on, and here's how I verified it properly" is itself a good thing to be able to speak to if asked how you validated your own approach.

8. Code Architecture

```
□
nnteg/
nnteg/
  __init__.py
  data.py    # fetch + filter + epoch + subject grouping
  model.py   # CSP + LDA pipeline
  evaluate.py # the six-part evaluation gauntlet
  train.py   # runs the full experiment, saves a trained model
  predict.py  # deliverable #2: CLI, raw EDF path -> predictions
  requirements.txt
```

data.py — downloads runs 4/8/12 for chosen subjects, bandpass filters to 7-30 Hz, epochs by T1/T2 annotation into snippets from -1.0 to 4.0 seconds relative to cue, crops the training window to 1.0-2.0s. Outputs `X` (trial snippets), `y` (left/right labels), `groups` (subject ID per trial — this is what makes leave-subject-out possible).

model.py — `build_pipeline()` returns the CSP(`n_components=4`)+LDA scikit-learn `Pipeline`. Packaging as one Pipeline object ensures CSP is applied identically at train and test time — applying it differently would itself be a leakage bug.

`evaluate.py` — six functions, one per check in Section 4, plus a `summarize()` that runs and reports all six together. Optionally includes an `align_by_subject(X, groups)` function implementing Section 5's alignment, off by default, callable as a toggle for comparison.

`train.py` — CLI: load subjects → run all six checks (and the alignment comparison, if time allows) → fit final model on everything → save to disk.

`predict.py` — deliverable #2. Loads the saved model, takes a raw EDF path, outputs left/right predictions, with pinned dependency versions and a one-line usage example. **This is the piece NT@B will actually execute on a subject you didn't use** — per general take-home-assessment research, a broken deliverable here is a fast rejection regardless of how strong the reasoning is elsewhere, so this gets real verification time, not an afterthought.

Note: the coding sandbox used to plan this cannot reach physionet.org (not on its allowed network list) — the code must be run locally where real internet access exists.

9. Open Decisions (Not Yet Finalized — Action Items, Not Research Gaps)

Exact subject count. Discussed as "medium to large"; 60 was floated as a concrete recommendation (leaves a meaningful held-out pool from the 109 available while keeping download/compute time reasonable) but never explicitly confirmed as final.

Leave-subject-out fold structure — how many subjects held out per fold, how many folds total. Affects how noisy the headline number will be; more held-out subjects per estimate = less noisy but fewer folds possible.

Local environment — Python version, whether MNE/scikit-learn are installed, which machine (MacBook referenced in mentor conversation). Not yet confirmed.

GitHub repo — not yet created; must be public per the brief.

requirements.txt pinning — exact package versions not yet written; required for `predict.py`'s deliverable spec.

Video recording — content is mapped (Section 10 below) but format/tool/rehearsal not yet done.

10. Video Content, Mapped Directly to the Brief's Required Bullets

"Your evaluation and what it establishes" — present all six numbers as a table, not just the best one. State the naive number, then immediately explain why it's inflated using the subject-identity leakage number (#5) as *evidence*, not just a claim.

"One design decision, a real alternative, and why you went the way you did" — CSP+LDA vs. a deep net (or CSP+LDA alone vs. CSP+LDA+alignment, if Section 5 gets built). State what would have been gained or lost either way.

"Your weakest point" — name a real one. Strong honest candidates: the leave-subject-out estimate is only as stable as the number of held-out subjects, so it carries real sampling noise; or the 7-30 Hz band and 1-2s training window were adopted from a reference implementation rather than independently validated against this specific data.

"Something we did not ask about" — the parallel between this project's core confound (inter-subject variance in baseline signal) and a prior HRV/panic-detection project's identical confound. This is a genuine observation from real prior work, not a forced tie-in — and it's not something another applicant could produce without that same background.

"What you would do next" — larger subject count, alignment explored further, EEGNet/ShallowFBCSPNet comparison.

"AI use, and one specific choice explained in your own words" — be honest about using AI to plan and scaffold. Pick one concrete decision (e.g., why leave-subject-out rather than random split) and explain it entirely without reading from notes — this is explicitly testing understanding, not recall.

11. Risk Summary

Risk	Mitigation
<code>predict.py</code> fails on a subject not used in training	Explicit Phase 4 verification step before considering the project done, not an afterthought
Time overrun past the 10-hour budget	Priority order in Section 5.3 — core pipeline first, alignment only if time remains, second architecture skipped entirely
Label-shuffle control doesn't collapse to ~50%	Signals a pipeline bug (likely a split leak or preprocessing leak) — treat as a debugging trigger, not a result to report as-is
Overstating confidence on any specific number or parameter	Everything in Sections 3.3 and 7 was checked against a real source before being finalized here; any new parameter introduced later should get the same treatment before being treated as settled

Sources Consulted During Planning

Original project brief (FA26 NT@B Software Division Recruitment Project PDF)

Project roadmap notes (ntb-eeg-project-roadmap.md)

Schalk et al., IEEE Trans. Biomed. Eng. 51(6):1034-1043 (2004) — original dataset paper (referenced, not independently re-read in full)

MNE-Python official CSP decoding example for the EEGBCI dataset (mne.tools, multiple version pages, consistent parameters across versions)

Multiple published papers on PhysioNet EEGMMIDB trial structure and classification (arXiv, PubMed, MDPI — cited inline in Sections 1.3 and 5.2)

General research on take-home technical assessment evaluation practices (used for the `predict.py` risk framing in Section 11)